

Εμπόδια για ένα Λάμα

Ένα λάμα θέλει να ταξιδέψει μέσα από το οροπέδιο των Άνδεων. Έχει έναν χάρτη του οροπεδίου με τη μορφή ενός πλέγματος που αποτελείται από $N \times M$ τετράγωνα κελιά. Οι γραμμές του χάρτη είναι αριθμημένες από 0 έως $N - 1$, από πάνω προς τα κάτω, και οι στήλες του είναι αριθμημένες από 0 έως $M - 1$, από τα αριστερά προς τα δεξιά. Το κελί του χάρτη στην γραμμή i και στήλη j (όπου $0 \leq i < N, 0 \leq j < M$) συμβολίζεται με (i, j) .

Το λάμα έχει μελετήσει το κλίμα του οροπεδίου και έχει ανακαλύψει ότι όλα τα κελιά σε κάθε γραμμή του χάρτη έχουν την ίδια **θερμοκρασία** και όλα τα κελιά σε κάθε στήλη του χάρτη έχουν την ίδια **υγρασία**. Το λάμα σάς έχει δώσει δύο πίνακες ακεραίων αριθμών T και H μήκους N και M αντίστοιχα. Το $T[i]$ (όπου $0 \leq i < N$) συμβολίζει την θερμοκρασία των κελιών στη γραμμή i , και το $H[j]$ (όπου $0 \leq j < M$) συμβολίζει την υγρασία των κελιών στη στήλη j .

Το λάμα έχει επίσης μελετήσει την χλωρίδα του οροπεδίου και παρατήρησε ότι ένα κελί (i, j) είναι **χωρίς βλάστηση** αν και μόνο αν η θερμοκρασία του είναι μεγαλύτερη από την υγρασία του, δηλαδή $T[i] > H[j]$.

Το λάμα μπορεί να κινηθεί εντός του οροπεδίου ακολουθώντας μόνο **έγκυρα μονοπάτια**. Ένα έγκυρο μονοπάτι είναι μια ακολουθία διακριτών (διαφορετικών μεταξύ τους) κελιών που ικανοποιούν τις ακόλουθες συνθήκες:

- Κάθε δύο διαδοχικά κελιά του μονοπατιού έχουν κοινή πλευρά.
- Όλα τα κελιά στο μονοπάτι είναι χωρίς βλάστηση.

Η δουλειά σας είναι να απαντήσετε Q ερωτήματα. Για κάθε ερώτημα, σας δίνονται τέσσερις ακέραιοι αριθμοί: L, R, S , και D . Πρέπει να αποφανθείτε κατά πόσον υπάρχει έγκυρο μονοπάτι τέτοιο ώστε να ισχύουν τα παρακάτω:

- Το μονοπάτι ξεκινάει από το κελί $(0, S)$ και τελειώνει στο κελί $(0, D)$.
- Όλα τα κελιά στο μονοπάτι βρίσκονται μεταξύ των στηλών L και R , συμπεριλαμβανομένων.

Είναι εγγυημένο ότι τα κελιά $(0, S)$ και $(0, D)$ είναι χωρίς βλάστηση.

Λεπτομέρειες Υλοποίησης

Η πρώτη συνάρτηση που πρέπει να υλοποιήσετε είναι:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : ένας πίνακας μήκους N που προσδιορίζει την θερμοκρασία κάθε γραμμής.
- H : ένας πίνακας μήκους M που προσδιορίζει την υγρασία κάθε στήλης.
- Αυτή η συνάρτηση καλείται ακριβώς μία φορά για κάθε περίπτωση ελέγχου, πριν από οποιαδήποτε κλήση της `can_reach`.

Η δεύτερη συνάρτηση που πρέπει να υλοποιήσετε είναι:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : ακέραιοι αριθμοί που περιγράφουν ένα ερώτημα.
- Αυτή η συνάρτηση καλείται Q φορές για κάθε περίπτωση ελέγχου.

Η συνάρτηση πρέπει να επιστρέφει `true` αν και μόνο αν υπάρχει έγκυρο μονοπάτι από το κελί $(0, S)$ στο κελί $(0, D)$, τέτοιο ώστε όλα τα κελιά του μονοπατιού να βρίσκονται μεταξύ των στηλών L και R , συμπεριλαμβανομένων.

Περιορισμοί

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ για κάθε i τέτοιο ώστε $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ για κάθε j τέτοιο ώστε $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Τα κελιά $(0, S)$ και $(0, D)$ είναι χωρίς βλάστηση.

Υποπροβλήματα

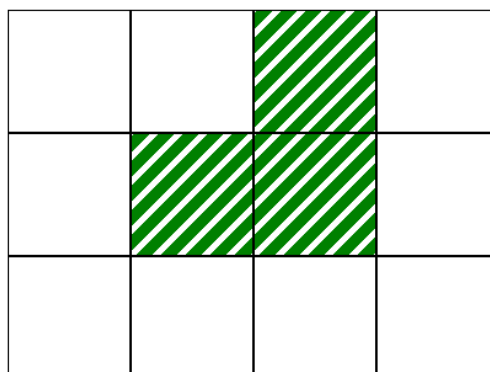
Υποπρόβλημα	Πόντοι	Επιπλέον περιορισμοί
1	10	$L = 0, R = M - 1$ για κάθε ερώτημα. $N = 1$.
2	14	$L = 0, R = M - 1$ για κάθε ερώτημα. $T[i - 1] \leq T[i]$ για κάθε i τέτοιο ώστε $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ για κάθε ερώτημα. $N = 3$ και $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ για κάθε ερώτημα. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ για κάθε ερώτημα.
6	17	Χωρίς επιπλέον περιορισμούς.

Παράδειγμα

Έστω η ακόλουθη κλήση:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

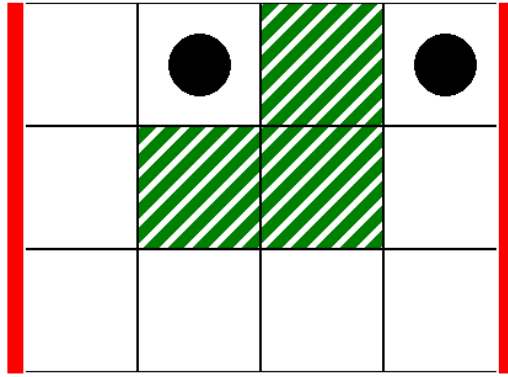
Αυτή η κλήση αντιστοιχεί στον χάρτη της ακόλουθης εικόνας, όπου τα άσπρα κελιά είναι χωρίς βλάστηση:



Έστω η ακόλουθη κλήση, ως πρώτο ερώτημα:

```
can_reach(0, 3, 1, 3)
```

Αυτό το ερώτημα αντιστοιχεί στο σενάριο της ακόλουθης εικόνας, όπου οι παχιές κάθετες γραμμές καθορίζουν το εύρος των στηλών από $L = 0$ έως $R = 3$, και οι μαύροι δίσκοι υποδεικνύουν το αρχικό και το τελικό κελί:



Σε αυτήν την περίπτωση, το λάμα μπορεί να φτάσει από το κελί (0,1) στο κελί (0,3) χρησιμοποιώντας το ακόλουθο έγκυρο μονοπάτι:

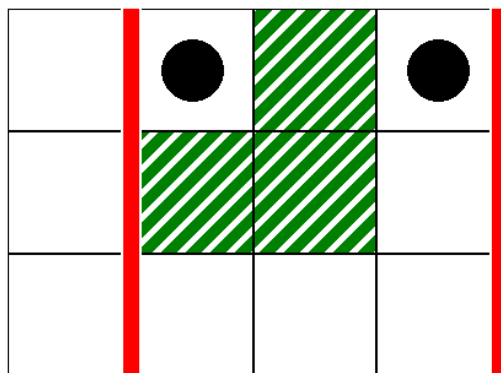
(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)

Επομένως, αυτή η κλήση πρέπει να επιστρέψει `true`.

Έστω η ακόλουθη κλήση, ως δεύτερο ερώτημα:

```
can_reach(1, 3, 1, 3)
```

Αυτό το ερώτημα αντιστοιχεί στο σενάριο της ακόλουθης εικόνας:



Σε αυτήν την περίπτωση, δεν υπάρχει έγκυρο μονοπάτι από το κελί (0,1) στο κελί (0,3), τέτοιο ώστε όλα τα κελιά στο μονοπάτι να βρίσκονται μεταξύ των στηλών 1 και 3, συμπεριλαμβανομένων. Επομένως, αυτή η κλήση πρέπει να επιστρέψει `false`.

Υποδειγματικός Βαθμολογητής

Μορφή εισόδου:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Εδώ, τα $L[k]$, $R[k]$, $S[k]$ και $D[k]$ (όπου $0 \leq k < Q$) προσδιορίζουν τις παραμέτρους για κάθε κλήση της `can_reach`.

Μορφή εξόδου:

```
A[0]
A[1]
...
A[Q-1]
```

Εδώ, το $A[k]$ (όπου $0 \leq k < Q$) ισούται με 1 εάν η κλήση της `can_reach(L[k], R[k], S[k], D[k])` επέστρεψε `true`, διαφορετικά ισούται με 0.